CAPEv2 Sandbox Master Deployment & Engineering Specification

DEFINITIVE PRODUCTION GUIDE: HARDENED NESTED VIRTUALIZATION, STEALTH KVM EVASION, POETRY ISOLATION, AND COMPLETE TELEMETRY

Document Version:	v2.1.0 (Sanitized Edition)	**Target Audience:**	Malware Researchers, Reverse Engineers, DevSecOps Teams
System Topology:	Nested Virtualization (Proxmox L0 → KVM Host L1 → Win10 Guest L2)	**Operational Profile:**	Advanced OpSec Evasion Hardened Environment

1. Architectural Design & Topology Mapping

Deploying a production-grade malware analysis framework within an enterprise virtualization infrastructure requires a strict multi-tier nested virtualization architecture. Standard sandbox virtualization platforms are easily identified by modern evasive malware via structural registry artifacts, hypervisor-specific instruction delays, and memory execution offsets. To circumvent detection, this guide enforces physical instruction passthrough from bare-metal hardware combined with kernel-level binary camouflage modifications to form an advanced Anti-VM architecture.

Infrastructure Layer	Platform Runtime Environment	Functional System Role Within Sandbox Lifecycle
Level 0 (Metal)	Proxmox VE (Physical Hypervisor Node)	Exposes raw hardware physical execution rings, memory MMU management, and registers nested hardware flags down to core guests.
Level 1 (VM Host)	Ubuntu Server 22.04 LTS (CAPE Controller)	Executes asynchronous celery management tracking engines, operates WireGuard/Tor routing engines, and drives custom compiled stealth hypervisors.
Level 2 (Nested VM)	Windows 10 Pro (Victim Analysis Node)	Exploded execution context node where malicious binaries interact with mock user environments. State changes are fully logged by monitoring drivers.

2. Phase 1: Preparing the Bare-Metal Proxmox Host (Level 0)

To ensure low-latency performance and provide hardware-assisted execution acceleration inside the intermediate guest layers, hardware virtualization extensions (Intel VT-x/VMX or AMD-V/SVM) must be explicitly unblocked at the root bare-metal kernel level.

2.1 Validation and Kernel Injections

Establish a secure root shell connection to your bare-metal Proxmox VE server node and assess the current initialization state of the kernel nested parameters:

```
# Interrogate the active hypervisor architecture flags
# For Intel Processor Subsystems:
cat /sys/module/kvm_intel/parameters/nested

# For AMD Processor Subsystems:
cat /sys/module/kvm_amd/parameters/nested
```

If the evaluation script reports an inactive flag (`N` or `0`), commit the persistent module mapping override loops required to forcefully assign full hardware nesting privileges:

```
# Execution commands for Intel Hardware Platforms:
echo "options kvm-intel nested=Y" | sudo tee /etc/modprobe.d/kvm-intel.conf
sudo modprobe -r kvm_intel
sudo modprobe kvm_intel

# Execution commands for AMD Hardware Platforms:
echo "options kvm-amd nested=1" | sudo tee /etc/modprobe.d/kvm-amd.conf
sudo modprobe -r kvm_amd
sudo modprobe kvm_amd
```

Re-Verification: Re-run the diagnostic evaluation commands above. The active return state must read precisely `Y` or `1` before scaling up to intermediate layers.

3. Phase 2: Deploying the Level 1 CAPE Controller Virtual Machine

The central management hub and virtualization host must run on a stable operational base. **Ubuntu Server 22.04 LTS** represents the verified long-term support release requirement for CAPEv2 dependencies. Avoid installing Ubuntu 24.04 at this time to eliminate baseline package discrepancies.

3.1 Proxmox Compute Definition Blueprint

Provision the new L1 Virtual Machine instance within the Proxmox environment matching the explicit operational thresholds defined below:

- **Processors & CPU Type:** `host` (CRITICAL STEP). Navigate to `VM → Hardware → Processors → Type` and select `Host` from the selection dialog. Alternatively, enforce it via the cluster command-line terminal:

  ```
  qm set <VMID> --cpu host --numa 1 --cores 8
  ```

  Why? This directly maps physical registers down into the Level 1 virtual wrapper, empowering the intermediate layer to initialize inner nested hardware acceleration boundaries.
- **Memory Allocation:** Allocation must be a minimum of `16384MB` (16GB) to safely maintain processing boundaries for the Ubuntu system daemonry plus multiple simultaneous Windows guest layers.
- **Storage Matrix:** Define a minimum size of `200GB` using a thin-provisioned layer or high-throughput storage network to provide deep staging space for PCAP dumps, binary extractions, and local MongoDB collections.
- **Network Interface:** Bind directly to your management network loop via a `VirtIO (Bridged)` card type linked to `vmbr0`.

3.2 Intermediate Layer Acceleration Audit

Power on the Level 1 Ubuntu VM host, establish local access, and confirm processing flags pass perfectly across the virtual container:

```
# Query internal hardware capability counters
egrep -c '(vmx|svm)' /proc/cpuinfo
```

> **Evaluation Rule:** If the returning command count displays `0`, the processor mapping loop is obstructed. Power off the host container and re-verify Phase 1 kernel settings. A value of `1` or higher validates total readiness for stealth hypervisor compilation.

4. Phase 3: The Source-Compiled "Stealth" KVM Engine (Anti-VM Evasion)

Stock virtualization packages distribute explicit tracking artifacts and generic descriptors that modern commercial malware uses to immediately cease behavior payloads. To systematically bypass these defensive routines, this architecture compiles a stripped and camouflaged fork of QEMU from low-level source files.

4.1 System Staging & Base Dependencies

Execute a global environment update loop and pull down the explicit compilation tooling chains required for lower-level source modification:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git build-essential python3-pip libffi-dev libssl-dev
```

4.2 Source-Tree Cloning & The "WOOT" Camouflage Technique

Clone the master CAPEv2 configuration and installer trees directly to the unified framework path:

```
cd /opt
sudo git clone https://github.com/CAPESandbox/CAPEv2.git
sudo chown -R $USER:$USER /opt/CAPEv2
cd /opt/CAPEv2/installer
```

Engineering the WOOT Camouflage Signature: The automated compilation script `kvm-qemu.sh` houses structural template markers labeled `<WOOT>`. These placeholders dictate the physical hardware tags, motherboard naming conventions, and system vendor descriptors broadcast by the hypervisor layer to inner guests.

To neutralize signature profiling, use a text editor to locate and replace the `<WOOT>` string definitions inside the installer files with authentic consumer corporate strings (e.g., `Intel Corporation`, `ALASKA`, or standard workstation vendor names). Once the values are updated, initialize the complete environment compiler loop:

```
chmod +x kvm-qemu.sh
# Execute the monolithic automated compiler loop
sudo ./kvm-qemu.sh all $USER
```

Note: This configuration phase parses, refactors, and builds QEMU binaries directly from raw source states. Execution times will vary significantly depending on CPU resource limits. Reboot the host VM container once execution terminates successfully.

5. Phase 4: Core Dependency Phase & Poetry Environment Isolation

Modern iterations of CAPEv2 completely discard unmanaged system-wide pip requirements lists in favor of **Python Poetry**. Poetry acts as a highly deterministic dependency management engine, using cryptographic lockfiles (`poetry.lock`) to enforce exact library versions. This prevents software drift from breaking the nested environment.

5.1 Poetry Global Deployment and Pathing

Before firing the foundational application logic configuration layer, deploy Python Poetry globally outside the unmanaged user spaces:

```
# Deploy the verified automated Poetry ecosystem installer
curl -sSL https://install.python-poetry.org | python3 -

# Bind the execution binary directly to the current system shell path context
export PATH="$HOME/.local/bin:$PATH"

# Persist path alignment permanently across all future administrative shell entries
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Validate deployment status
poetry --version
```

5.2 Framework Initialization Pipeline

With Poetry successfully integrated, enter the staging directory to compile the baseline application infrastructure and map framework user environments:

```
cd /opt/CAPEv2/installer
chmod +x cape2.sh

# Initialize the infrastructure baseline and component compiler
sudo ./cape2.sh base cape
sudo reboot
```

6. Phase 5: Provisioning the Level 2 Windows Victim Guest Node

The targeted detonator node operates within an isolated virtual switch segment. To create the image container inside the Level 1 Ubuntu VM host, leverage the native `virt-manager` graphic orchestration console.

6.1 VM Deployment & Installation Framework

Establish a terminal session into the Ubuntu host using X11 forwarding privileges or interface through an accessible remote desktop shell environment:

```
sudo virt-manager
```

Construct a clean virtual instance passing the following strict environment arguments:

- **Machine Descriptor Name:** `win10_native`

- **Storage Profile:** Must use the `QCOW2` disk image configuration format (Required to handle iterative live state rollback operations and differential snapshots).
- **Networking Interface Assignment:** Bind the network wrapper profile directly to the internal isolated segment created during automated staging (e.g., `virbr1` or your dedicated internal malware communication bridge).

6.2 Victim Guest Camouflage and Hardening Matrix

Complete the standard Windows 10 installation sequence. Once operating inside the target desktop workspace, execute the mandatory hardening tasks below to suppress administrative background checks and monitoring alerts:

1. **Deactivate Endpoint Defenses:** Completely disable real-time analysis modules and Windows Defender systems. Enforce these blocks permanently via administrative Group Policy rules (GPO) or leverage automated third-party utilities like *Defender Control*.
2. **Suppress System Firewalls:** Access the advanced security management control terminal and toggle all public, private, and domain firewall profiles to *Disabled*.
3. **Deactivate Automated Updates:** Set the baseline startup parameters for the Windows Update service component to *Disabled* and terminate the active thread.
4. **Deploy Python Environment:** Download and install **Python 3.10 (32-bit/x86 version)** inside the Windows guest environment. Check the validation prompt to explicitly add the target executable mapping to the global system `PATH` variable.

6.3 Monitoring Agent Injection Protocols

The CAPEv2 control layer communicates directly with the victim node via a secure management agent script loop.

```
# Reference Location on the Level 1 Ubuntu Host:
# /opt/CAPEv2/agent.py
```

Transfer `agent.py` into the target Windows guest system space. Rename the file extension to exactly ** `agent.pyw` **. The `.pyw` assignment forces the Python runtime layer to execute the background script silently without spinning up a visible command prompt container window.

Inject the `agent.pyw` script map directly into the native startup directory by executing `shell:startup` via the Windows Run dialog, or provision it as an independent elevated Task Scheduler action configured to execute with administrative privileges on system initialization.

6.4 Secure Guest Static Networking Assignment

Malicious binaries require network access to fetch secondary execution payloads and check command-and-control connectivity. To prevent infrastructure contamination while ensuring proper telemetry capture, enforce strict IP assignments on the internal host network:

- **Target Guest Static IP:** `192.168.122.10`
- **Subnet Network Mask:** `255.255.255.0`
- **Default Network Gateway:** `192.168.122.1`
- **Upstream Domain Name Server (DNS):** `8.8.8.8`

6.5 Establishing the Base Snapshot State

Allow the Windows environment to complete all initial background processing loops until CPU metrics drop to a total rest state.

Verify that the agent script is successfully bound and actively listening on port `8000` via a command terminal (e.g., `netstat -ano | findstr 8000`). Access the parent `virt-manager` control screen and issue a cold snapshot execution. Assign the identifier string precisely as ** `Snap1` **.

7. Network Orchestration: Hardened Routing & Tor Proxy De-confliction

To maintain total operational security, all analysis traffic must be strictly routed through an encrypted out-of-band tunnel network interface or anonymized proxy topology.

7.1 Configuration of the WireGuard Isolation Envelope

Commit your upstream commercial exit node settings to `/etc/wireguard/wg0.conf` and verify that the tunnel starts up cleanly during multi-user environment initialization:

```
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0
sudo wg show wg0
```

7.2 Resolving the Tor Multi-Binding Collision Bug

During standard staging tests, an operational error frequently prevents Tor from initializing its interception endpoints.
Root Cause: Overlapping duplicate configuration loops in `/etc/tor/torrc` frequently assign explicit listener interfaces to static addresses while simultaneously specifying broad `0.0.0.0` bindings, causing a fatal socket collision crash (*Address already in use*).

Scrub the contents of `/etc/tor/torrc` completely and map out the sanitized, non-colliding transport profile block below:

```
[torrc]
NumCPUs 4
SocksTimeout 60
ControlPort 9051

VirtualAddrNetworkIPv4 10.192.0.0/10
AutomapHostsOnResolve 1

TransPort 0.0.0.0:9040
DNSPort 0.0.0.0:5353
```

Commit the changes by restarting the service daemon, and run verification tools to ensure that both the transparent proxy and DNS structures are actively listening on all local interfaces:

```
sudo systemctl restart tor
sudo ss -tulpn | grep -E '9040|5353'
```

7.3 Enforcing the Network Firewall Kill-Switch

To prevent a malformed tracking state or drop in the WireGuard interface from leaking traffic out of your raw physical gateway interface (`eth0`), inject an immutable firewall forward drop rule:

```
sudo iptables -I FORWARD -i virbr0 -o eth0 -j DROP
sudo apt install iptables-persistent
```

8. Configuration Mapping & Scheduler Optimization

To tie the core orchestration system to your new stealth virtual machine environments, modify the primary framework settings maps located under the central configuration directories.

8.1 Tuning cuckoo.conf

Open `/opt/CAPEv2/conf/cuckoo.conf` and assign the primary hypervisor machinery driver to KVM while defining the tracking analysis callback interface:

```
[cuckoo]
machinery = kvm

[resultserver]
# The IP address corresponding to your Level 1 Ubuntu VM internal bridge gateway interface
ip = 192.168.122.1
port = 2042
```

8.2 Repairing the "Unserviceable Task" Scheduling Bug

When processing complex analysis runs, tasks frequently freeze inside tracking loops displaying the fatal error string: `INFO: Task #9: Failing unserviceable task because no matching machine could be found. Requested tags: 'x64'`.
Root Cause Analysis: Configuration files frequently contain architecture mismatch strings (e.g., `arch = x86`) or include hidden trailing whitespace artifacts (e.g., `tags = windows_10, x64 `). Because CAPE processes parameters literally, it evaluates trailing spaces as string data, rendering lookups invalid.

Open `/opt/CAPEv2/conf/kvm.conf`, purge trailing spaces completely, and map out the exact configuration properties block:

```
[kvm]
machines = win10_native

[win10_native]
label = win10_native
platform = windows
ip = 192.168.122.10
arch = x64
tags = windows_10,x64
snapshot = Snap1
interface = virbr0
```

9. Advanced Forensic Instrumentation & Telemetry

To maximize indicators of compromise (IoC) collection efficiency, enable automated deep forensic tracking within `/opt/CAPEv2/conf/processing.conf` and `/opt/CAPEv2/conf/routing.conf`.

9.1 Suricata NIDS Mapping

```
[suricata]
enabled = yes
bin = /usr/bin/suricata
config = /etc/suricata/suricata.yaml
```

9.2 YARA Compilation Protocols

When mapping custom detection rules into memory scanners, always make sure the rule files are registered directly inside the root indexing schema:

```
# 1. Output custom file path definitions:
# /opt/CAPEv2/data/yara/memory/my_custom_rules.yar

# 2. Append directly to the mandatory index configuration file:
# /opt/CAPEv2/data/yara/memory/index_memory.yar
include "my_custom_rules.yar"
```

9.3 Volatility 3 Kernel RAM Forensic Profiles

To perform automated memory extraction sequences before tearing down analysis environments, activate full memory dump loops inside `cuckoo.conf` and configure your targets inside `memory.conf`:

```
# cuckoo.conf mapping parameter
memory_dump = yes

# memory.conf active monitoring target switches
[pslist]
enabled = yes
[pstree]
enabled = yes
[malfind]
enabled = yes
```

10. Administrative Lifecycles & Operations

Whenever adjustments are made to internal configurations, rule listings, or networking parameters, cycle the core environment processes using systemd commands to ensure clean integration:

```
# Commit changes and recycle background execution daemons
sudo systemctl restart cape
sudo systemctl restart cape-processor
sudo systemctl restart cape-web

# Confirm runtime execution status across processing interfaces
sudo systemctl status cape cape-processor --no-pager -l
```

Open your local web browser and navigate directly to your interface portal at `http://<YOUR_UBUNTU_IP>:8000`. Submit a benign utility to verify the deployment state. If the target Windows VM wakes up automatically, processes the binary payload, and resets cleanly, the installation is fully complete.